Submitted By

Samay Raj Meena

Roll No: **240919**

Domain:	EEG Signal Processing, BCI, Machine Learning
Dataset:	BNCI2014-009 (EPFL P300 Speller) via MOABB
Subjects:	10 healthy participants
Models:	LDA, SVM, EEGNet (TensorFlow)
Date:	7 April 2026

Contents

1 Abstract **3**

2 Introduction **3**
2.1 Background and Motivation . . . 3
2.2 The P300 Paradigm . . . 4
2.3 Objectives . . . 4

3 Literature Review **5**
3.1 P300 Speller — Farwell & Donchin (1988) . . . 5
3.2 EEGNet — Lawhern et al. (2018) . . . 5
3.3 Classification Survey — Lotte et al. (2018) . . . 5
3.4 MOABB — Jayaram & Barachant (2018) . . . 5

4 Dataset **5**
4.1 BNCI2014-009 (EPFL P300 Speller) . . . 5
4.2 Data Loading via MOABB . . . 6
4.3 Class Imbalance . . . 6

5 Signal Processing Pipeline **7**
5.1 Overview . . . 7
5.2 Bandpass Filtering . . . 7
5.3 Notch Filter . . . 7
5.4 Epoch Extraction and Baseline Correction . . . 7
5.5 Independent Component Analysis (ICA) . . . 8
5.6 Event-Related Potential (ERP) . . . 8

6 Feature Extraction **8**
6.1 Downsampling + Flattening (Baseline) . . . 8
6.2 Feature Scaling . . . 8

7 Classification Models **9**
7.1 Linear Discriminant Analysis (LDA) . . . 9
7.2 Support Vector Machine (SVM) . . . 9
7.3 EEGNet (Deep Learning) . . . 10
7.3.1 Architecture . . . 10

8 Evaluation Methodology **10**

8.1 Cross-Validation . . . 10
8.2 Train / Test Split . . . 11
8.3 Metrics . . . 11
8.3.1 Classification Accuracy . . . 11
8.3.2 AUC-ROC . . . 11
8.3.3 Information Transfer Rate (ITR) . . . 11
8.4 Character-Level Decoding . . . 11

9 Results **11**
9.1 5-Fold Cross-Validation (Training Set) . . . 12
9.2 Model Comparison on Test Set . . . 12
9.3 Key Observations . . . 12

10 Discussion **13**
10.1 Signal Processing Choices . . . 13
10.2 Feature Engineering vs. Deep Learning . . . 13
10.3 Class Imbalance Handling . . . 13
10.4 Computational Setup and Limitations . . . 14

11 Conclusion **14**
11.1 Future Work . . . 14

References **15**

1. Abstract

Summary

This report presents the design, implementation, and evaluation of an EEG-based Brain-Computer Interface (BCI) system — the **Brain Speller**. The system enables a user to type characters using only their brainwave signals with no physical movement required. It operates on the P300 Speller paradigm, where a 6×6 matrix of characters is presented on screen and rows/columns flash in a random sequence.

The complete pipeline encompasses: data acquisition via the **BNCI2014-009** dataset (10 subjects) downloaded through MOABB; signal preprocessing including a 0.1–20 Hz bandpass filter, a 50 Hz notch filter, and Independent Component Analysis (ICA) for artefact rejection; epoch extraction locked to stimulus onsets; and feature extraction via temporal downsampling.

Two classical classifiers — **Linear Discriminant Analysis (LDA)** and **Support Vector Machine (SVM)** — are evaluated using 5-fold stratified cross-validation. A deep learning model, **EEGNet**, is additionally implemented in TensorFlow. All models are evaluated using classification accuracy, AUC-ROC, and Information Transfer Rate (ITR) in bits per minute.

2. Introduction

2.1 Background and Motivation

Brain-Computer Interfaces (BCIs) represent a transformative technology that creates a direct communication channel between the human brain and external devices. Unlike conventional interfaces that require muscle movement, BCIs decode neural signals directly from the brain, offering life-changing possibilities for individuals with severe motor disabilities such as ALS (Amyotrophic Lateral Sclerosis), spinal cord injuries, and locked-in syndrome.

The **P300 Speller**, first introduced by Farwell and Donchin in 1988 [1], remains one of the most widely studied and clinically viable BCI paradigms. It exploits the brain's natural **P300 Event-Related Potential (ERP)** — a positive deflection in EEG signal peaking around 300 ms after a rare, attended stimulus — to identify which character a

user intends to select.

2.2 The P300 Paradigm

A 6×6 character matrix is displayed on screen. Rows and columns flash one at a time in a random order. When the *target character's* row or column flashes, the brain generates a distinctive P300 response. Non-target flashes produce a flat EEG signal. The classifier scores all 12 rows/columns and selects the row and column with the highest P300 probability — their intersection gives the predicted character.

```
Character Matrix (6 x 6)          P300 ERP Response

  A  B  C  D  E  F          Amplitude (µV)
  G  H  I  J  K  L              ^
  M  N  O  P  Q  R              |    /\  <- P300 (~300ms)
  S  T  U  V  W  X  Target  |   /  \
  Y  Z  1  2  3  4              |/
  5  6  7  8  9  _  Non-Target  flat line

      ^--- Column flash                 0  100 200 300 400 500ms
```

Figure 1: P300 Speller paradigm: character matrix and expected ERP waveforms.

2.3 Objectives

1. Download and preprocess real multi-subject EEG data from the BNCI2014-009 P300 dataset via MOABB.
2. Implement a complete signal processing pipeline: bandpass filtering, notch filtering, ICA artefact rejection, and epoch extraction.
3. Extract discriminative features using temporal downsampling.
4. Train and evaluate classical classifiers (LDA and SVM) using stratified k-fold cross-validation.
5. Implement **EEGNet** — a compact CNN designed for EEG classification.
6. Evaluate all models using Accuracy, AUC-ROC, and ITR.

3. Literature Review

3.1 P300 Speller — Farwell & Donchin (1988)

The foundational work by Farwell and Donchin [1] introduced the concept of using the P300 ERP for character selection. Their system presented a 6×6 matrix of alphanumeric characters and flashed rows and columns. The brain's response to the target character's row or column flash produced a distinguishable P300 wave. Their original system achieved approximately 95% accuracy with 2.3 characters per minute, demonstrating proof of concept for non-muscular communication.

3.2 EEGNet — Lawhern et al. (2018)

Lawhern et al. [2] proposed EEGNet, a compact convolutional neural network architecture specifically designed for EEG-based BCIs. EEGNet generalises across multiple BCI paradigms (P300, SSVEP, Motor Imagery) using a small number of parameters. It uses depthwise and separable convolutions to efficiently learn temporal and spatial EEG features, achieving strong performance with minimal data — making it ideal for this project.

3.3 Classification Survey — Lotte et al. (2018)

Lotte et al. [3] conducted a comprehensive review of classification algorithms for EEG-based BCIs spanning over a decade. Key findings:

- LDA remains a strong and interpretable baseline.
- SVM with RBF kernel performs well on high-dimensional EEG features.
- Riemannian geometry classifiers offer state-of-the-art performance.
- Deep learning requires large datasets but can surpass classical approaches.

3.4 MOABB — Jayaram & Barachant (2018)

The Mother of All BCI Benchmarks (MOABB) [4] provides a standardised framework for evaluating BCI pipelines across datasets and subjects. MOABB handles dataset downloading, paradigm-specific epoch extraction, and consistent cross-validation splits, enabling reproducible comparisons.

4. Dataset

4.1 BNCI2014-009 (EPFL P300 Speller)

The dataset used in this project is **BNCI2014-009**, accessed programmatically via the MOABB framework. This eliminates manual data downloading and provides standardised

epoch extraction.

Table 1: BNCI2014-009 Dataset Specifications

Property	Details
Number of Subjects	10 healthy participants
Paradigm	P300 Speller (6×6 character matrix)
EEG Channels	16 channels used in execution
Sampling Rate	256 Hz
Epoch Window	-200 ms to $+800$ ms around stimulus onset
Flash Duration	≈ 125 ms per flash
Flashes per Character	12 (6 rows + 6 columns)
Repetitions	10 repetitions per character
Access Method	`MOABB: BNCI2014_009()`

4.2 Data Loading via MOABB

```python
from moabb.datasets import BNCI2014_009
from moabb.paradigms import P300

dataset  = BNCI2014_009()
paradigm = P300(fmin=0.1, fmax=20.0, tmin=-0.2, tmax=0.8,
    baseline=(-0.2, 0.0))

# Load Subject 1  (returns epochs, labels, metadata)
X, y, meta = paradigm.get_data(dataset=dataset, subjects=[1])
# X shape: (n_epochs, n_channels=16, n_times=257)
# y: array of 'Target' / 'NonTarget' strings
y_binary = (y == 'Target').astype(int)   # convert to 0/1
```

Listing 1: Loading subject data using MOABB P300 paradigm

4.3 Class Imbalance

The P300 paradigm inherently creates a severe class imbalance. In each character trial, 12 stimuli are presented but only 2 (1 row + 1 column) correspond to the target character — a **1:5 target-to-non-target ratio**. This is addressed via:

- `class_weight='balanced'` in the SVM classifier.

- Proportional class weights in EEGNet training.
- Reporting AUC-ROC alongside accuracy as the primary metric.

5. Signal Processing Pipeline

5.1 Overview

The preprocessing pipeline transforms raw noisy EEG recordings into clean, labelled epochs suitable for machine learning. Each stage is applied sequentially.

Table 2: Signal Processing Pipeline Summary

Stage	Method	Parameters
Bandpass Filter	Butterworth IIR	0.1–20 Hz, order 5, zero-phase
Notch Filter	IIR Notch	50 Hz, $Q = 30$
ICA	FastICA	10 components
Epoch Extraction	Onset detection	-200 to $+800$ ms, 257 samples
Baseline Period	Pre-stimulus	-200 to 0 ms
Output Epoch Shape	Per subject	$(n_epochs,\ 16,\ 257)$

5.2 Bandpass Filtering

A 5th-order Butterworth bandpass filter is applied with a passband of **0.1 to 20 Hz**. The P300 component has its dominant spectral energy below 20 Hz. The lower cutoff of 0.1 Hz removes slow drift and DC offset. `filtfilt` achieves zero-phase filtering, preserving the temporal structure of the ERP.

5.3 Notch Filter

A 50 Hz IIR notch filter (Q = 30) removes power-line interference. In India, the AC mains frequency is 50 Hz, commonly picked up as a strong sinusoidal artefact by EEG electrodes.

5.4 Epoch Extraction and Baseline Correction

EEG epochs are extracted by detecting flash onset events ($0 \rightarrow 1$ transitions in the stimulus timing signal). Each epoch spans from -200 ms (pre-stimulus baseline) to $+800$ ms post-

stimulus. Baseline correction is applied using the -200 to 0 ms window to center the signal relative to the stimulus onset.

5.5 Independent Component Analysis (ICA)

To ensure clean neural signals for the classifiers, Independent Component Analysis (via `mne.preprocessing.ICA`) was fitted to the epoched data. Ten components were utilized to isolate and systematically remove prevalent non-cerebral artefacts, such as eye blinks and muscular movements, before feature extraction.

5.6 Event-Related Potential (ERP)

The P300 ERP is visualised by averaging all target and non-target epochs separately. The expected outcome is a positive peak around 250–400 ms post-stimulus, most prominent at central–parietal electrode locations.

6. Feature Extraction

To transform the 3D epoch arrays (epochs $\times$ channels $\times$ time) into a 2D feature matrices (epochs $\times$ features) suitable for classical machine learning, a Downsampling and Flattening approach was utilized.

6.1 Downsampling + Flattening (Baseline)

The time axis of each epoch is uniformly subsampled from 257 time points to 30, then all channels are concatenated into a single feature vector:

$$\mathbf{x} \in \mathbb{R}^{16\times 30=480}$$

```python
def downsample_flatten(X, target_samples=30):
    n_epochs, n_ch, n_times = X.shape
    idx = np.linspace(0, n_times-1, target_samples, dtype=int)
    return X[:, :, idx].reshape(n_epochs, -1)    # (n, 480)
```

Listing 2: Downsample and flatten feature extraction

6.2 Feature Scaling

All features are standardised using `StandardScaler` (zero mean, unit variance) after extraction. This is critical for SVM (RBF kernel uses Euclidean distances) and aids overall model convergence.

7. Classification Models

7.1 Linear Discriminant Analysis (LDA)

LDA is a generative classifier that models class-conditional distributions as multivariate Gaussians with a shared covariance matrix. The SVD solver avoids numerical issues. LDA is chosen as the primary baseline because it is fast, interpretable, and historically performs well on P300 EEG data.

```
from sklearn.discriminant_analysis import
   LinearDiscriminantAnalysis
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

lda = Pipeline([
    ('scaler', StandardScaler()),
    ('lda',    LinearDiscriminantAnalysis(solver='svd'))
])
lda.fit(X_train, y_train)
```

Listing 3: LDA classifier pipeline

7.2 Support Vector Machine (SVM)

SVM with an RBF kernel enables non-linear decision boundaries. Key settings:

- `class_weight='balanced'` — upweights target epochs to handle 1:5 imbalance.
- `probability=True` — enables Platt scaling for probability estimates used in character decoding.

```
from sklearn.svm import SVC

svm = Pipeline([
    ('scaler', StandardScaler()),
    ('svm',    SVC(kernel='rbf', probability=True,
                   class_weight='balanced', C=1.0))
])
```

Listing 4: SVM classifier with class balancing

7.3 EEGNet (Deep Learning)

EEGNet [2] is a compact CNN operating directly on raw epochs without manual feature extraction. Input shape: (batch, 16, 257, 1).

7.3.1 Architecture

EEGNet Block Structure

Block 1 — Temporal + Spatial: `Conv2D`($F_1 = 8,\ k = (1, 64)$) $\rightarrow$ `BatchNorm` $\rightarrow$ `DepthwiseConv2D`($D = 2$) $\rightarrow$ `BatchNorm` $\rightarrow$ `ELU` $\rightarrow$ `AvgPool`$(1, 4)$ $\rightarrow$ `Dropout`(0.5)

Block 2 — Feature Combination: `SeparableConv2D`($F_2 = 16,\ k = (1, 16)$) $\rightarrow$ `BatchNorm` $\rightarrow$ `ELU` $\rightarrow$ `AvgPool`$(1, 8)$ $\rightarrow$ `Dropout`(0.5)

Classifier: `Flatten` $\rightarrow$ `Dense`(2, softmax)

Table 3: EEGNet Layer-by-Layer Summary

Layer	Operation	Output Shape	Params
Input	EEG epochs	(16, 257, 1)	—
Conv2D ($F_1 = 8$)	Temporal conv, kernel $(1, 64)$	(16, 257, 8)	512
DepthwiseConv2D	Spatial filter, kernel $(16, 1)$, $D = 2$	(1, 257, 16)	256
AvgPool+Dropout	Pool $(1, 4)$, $p = 0.5$	(1, 64, 16)	—
SeparableConv2D	Pointwise conv, $F_2 = 16$, $(1, 16)$	(1, 64, 16)	512
AvgPool+Dropout	Pool $(1, 8)$, $p = 0.5$	(1, 8, 16)	—
Dense (Softmax)	Classification head, 2 classes	$(2,)$	258
		Total Trainable Parameters	**1,618**

8. Evaluation Methodology

8.1 Cross-Validation

5-fold **stratified** cross-validation is used on the training set. Stratification preserves the Target/Non-Target ratio in each fold. Scores are reported as mean ± standard deviation

across folds.

8.2 Train / Test Split

An 80/20 stratified split holds out a test set for final evaluation. The test set is never seen during training or cross-validation.

8.3 Metrics

8.3.1 Classification Accuracy

Proportion of correctly classified epochs. Due to the 1:5 imbalance, accuracy alone is insufficient — a trivial all-Non-Target classifier achieves $\approx 83\%$.

8.3.2 AUC-ROC

The Area Under the Receiver Operating Characteristic Curve measures the classifier's ability to rank target epochs above non-target epochs, independent of the decision threshold. AUC $= 1.0$ is perfect; AUC $= 0.5$ is random. This is the **primary metric** for imbalanced scenarios.

8.3.3 Information Transfer Rate (ITR)

ITR combines accuracy and speed into a single communication throughput measure:

$$\mathrm{ITR} = \left[\log_2 N + P \log_2 P + (1-P)\log_2\left(\frac{1-P}{N-1}\right)\right] \times \frac{60}{T} \quad [\text{bits/min}] \tag{1}$$

where $N = 36$ (symbols in the 6×6 grid), P is classification accuracy (0–1), and T is trial duration in seconds. Trial duration is estimated as 10 repetitions $\times$ 12 flashes $\times$ 0.125 s $=$ 15 s.

8.4 Character-Level Decoding

For each character trial, the classifier scores all 12 row/column flashes across all repetitions. Scores are averaged per row and per column. The row and column with the highest average P300 probability are selected, and their intersection gives the predicted character.

9. Results

9.1 5-Fold Cross-Validation (Training Set)

Table 4: Cross-validation accuracy (%) on training partition — mean ± std across 5 folds

Subject	LDA Down.	SVM Down.
S01	$89.0\% \pm 2.0\%$	$84.2\% \pm 1.7\%$
S02	$90.7\% \pm 2.2\%$	$88.5\% \pm 1.4\%$
S03	$85.6\% \pm 2.4\%$	$86.1\% \pm 1.8\%$
S04	$89.2\% \pm 1.2\%$	$88.3\% \pm 1.7\%$
S05	$91.0\% \pm 1.2\%$	$89.4\% \pm 2.0\%$
S06	$86.6\% \pm 2.0\%$	$82.6\% \pm 1.1\%$
S07	$87.7\% \pm 1.7\%$	$83.9\% \pm 1.4\%$
S08	$87.0\% \pm 1.3\%$	$85.2\% \pm 2.3\%$
S09	$91.8\% \pm 2.2\%$	$90.0\% \pm 2.4\%$
S10	$92.5\% \pm 1.5\%$	$92.7\% \pm 1.5\%$
Average	$\mathbf{89.1\% \pm 1.8\%}$	$\mathbf{87.1\% \pm 1.7\%}$

9.2 Model Comparison on Test Set

Table 5: Test-set performance comparison across all models (averaged over 10 subjects)

Model	Input Type	Accuracy	AUC	ITR (bits/min)
LDA	Downsampled Features	89.94%	0.9240	16.78
SVM (RBF)	Downsampled Features	87.75%	0.8876	16.04
EEGNet	Raw Epochs	90.06%	–	16.82

9.3 Key Observations

- **EEGNet vs Classical:** EEGNet achieved the highest overall test accuracy (90.06%) and ITR (16.82 bits/min), slightly outperforming the classical baselines. This demon-

strates the viability of end-to-end deep learning, as EEGNet learned directly from raw epochs without requiring the manual downsampling needed by LDA and SVM.

- **LDA vs SVM:** Interestingly, LDA (89.94% average accuracy) outperformed SVM (87.75% average accuracy) on the test sets. While SVM utilized explicit class weighting to counter the strict 1:5 Target-to-Non-Target imbalance, LDA's simple generative approach proved highly robust on the downsampled temporal features.
- Significant **inter-subject variability** exists: Subjects such as S09 and S10 consistently yielded excellent performance (achieving $> 94\%$ accuracy across multiple models), whereas others like S03 and S08 were significantly harder to decode (hovering around $83\% - 85\%$ accuracy).
- More flash **repetitions** improve accuracy by averaging out noise — 10 repetitions provide a good accuracy/speed tradeoff.

10. Discussion

10.1 Signal Processing Choices

The 0.1–20 Hz bandpass range was chosen specifically for the P300 paradigm. The P300 component has dominant energy between 0.5 and 8 Hz, but extending to 20 Hz preserves faster ERP components that contribute discriminative information. The 50 Hz notch is essential for Indian recordings where AC power runs at 50 Hz. Furthermore, wrapping the NumPy arrays back into MNE structures to run ICA proved necessary for stabilizing the classifier inputs against artefact contamination.

10.2 Feature Engineering vs. Deep Learning

The comparison between manual features (Downsampling + SVM) and end-to-end deep learning (EEGNet) reveals an important tradeoff: manual features provide more interpretable, data-efficient solutions that perform reliably with small per-subject datasets. EEGNet performs comparably but benefits from larger training sets and computational resources.

10.3 Class Imbalance Handling

The 1:5 target/non-target imbalance is addressed by three complementary strategies: (1) `class_weight='balanced'` in SVM; (2) proportional class weights in EEGNet training; and (3) reporting AUC-ROC alongside accuracy. Without these measures, a naive all-Non-Target classifier appears to achieve $\approx 83\%$ accuracy while being completely useless as a

BCI.

10.4 Computational Setup and Limitations

- **Hardware:** Model training and pipeline execution were conducted locally utilizing a Dell environment equipped with an Intel Core i5-1135G7 .Due to relying entirely on CPU computation, the ability to achieve these results demonstrates the high efficiency of the EEGNet architecture on standard consumer-grade hardware.
- Models trained and tested on the same session may overestimate real-world cross-day performance.
- The Psychopy stimulus interface is a demonstration only and has not been validated in a live real-time BCI session.

11. Conclusion

This project successfully implements a complete EEG-based P300 Brain Speller pipeline. Starting from raw EEG data downloaded via MOABB, the system performs signal preprocessing, feature extraction, and classification using both classical machine learning and deep learning approaches.

Key findings:

1. EEGNet proved to be the most robust model overall, achieving the highest average accuracy (90.06%) and ITR (16.82 bits/min), closely followed by the LDA baseline (89.94% accuracy).
2. Despite the severe 1:5 Target-to-Non-Target imbalance, simple generative models like LDA are highly effective when applied to properly downsampled temporal features.
3. EEGNet achieves top-tier performance completely end-to-end, bypassing the need for manual feature engineering.
4. Information Transfer Rates (ITR) ranging from 14.6 to 18.9 bits/min are achievable depending on the subject, which is highly consistent with published P300 BCI benchmarks.

11.1 Future Work

- Implement Riemannian geometry classifiers (`pyriemann`) using spatial covariance matrices to push for potential state-of-the-art accuracy.
- Explore transfer learning across subjects to reduce per-subject calibration time.

- Develop an adaptive repetition scheme that stops flashing once the classifier is sufficiently confident, further improving ITR.
- Deploy a live real-time P300 speller using Psychopy and a real EEG headset.

References

References

[1] L.A. Farwell and E. Donchin, "Talking off the top of your head: toward a mental prosthesis utilizing event-related brain potentials," *Electroencephalography and Clinical Neurophysiology*, vol. 70, no. 6, pp. 510–523, 1988.

[2] V.J. Lawhern, A.J. Solon, N.R. Waytowich, S.M. Gordon, C.P. Hung, and B.J. Lance, "EEGNet: A compact convolutional neural network for EEG-based brain-computer interfaces," *Journal of Neural Engineering*, vol. 15, no. 5, p. 056013, 2018.

[3] F. Lotte, L. Bougrain, A. Cichocki, M. Clerc, M. Congedo, A. Rakotomamonjy, and F. Yger, "A review of classification algorithms for EEG-based brain-computer interfaces: a 10+ year update," *Journal of Neural Engineering*, vol. 15, no. 3, p. 031005, 2018.

[4] V. Jayaram and A. Barachant, "MOABB: trustworthy algorithm benchmarking for BCIs," *Journal of Neural Engineering*, vol. 15, no. 6, p. 066011, 2018.

[5] B. Rivet, A. Souloumiac, V. Attina, and G. Gibert, "xDAWN algorithm to enhance evoked potentials: application to brain-computer interface," *IEEE Transactions on Biomedical Engineering*, vol. 56, no. 8, pp. 2035–2043, 2009.

[6] A. Gramfort et al., "MEG and EEG data analysis with MNE-Python," *Frontiers in Neuroscience*, vol. 7, p. 267, 2013.

[7] Y. Wang, X. Chen, X. Gao, and S. Gao, "A benchmark dataset for SSVEP-based brain-computer interfaces," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 25, no. 10, pp. 1746–1752, 2017.